# Mainframe Codebase Auditor — Project Plan

## 1. Overview

This project is a mainframe-powered batch processing pipeline that scans a GitHub repository's entire codebase (instead of plain CSV/data files) and produces a report describing:

- What technologies/frameworks are used in the codebase
- Whether each technology is genuinely integrated into the application logic, or just present for demo/pseudo purposes (e.g. React imported but unused)
- Summary statistics: duplicate files, record counts, technology usage breakdown

The core constraint from the course: **JCL, COBOL, and REXX must be used for their genuine strengths**, not as decoration. The plan below maps each of the four mandatory course requirements onto a real pipeline stage.

---

## 2. High-Level Architecture

Two systems running side by side on the same demo machine, talking through shared files (not a single tightly-coupled app):

1. **Plotoris (preprocessing layer)** — a Node.js/Bun-based agentic CLI tool (already built by Sarthak, a Claude-Code-style clone using an AI SDK) that:
    - Walks the target GitHub repository
    - Extracts cheap metadata per file with pure scripting (no AI calls): file path, extension, line count, content hash
    - Selectively calls an AI model (via OpenRouter) only for judgment-call questions that scripting can't answer, e.g. "is this React import actually wired into the app, or just present for show?"
        - Batches multiple files per AI call where possible, and only invokes AI on a filtered subset (e.g. files importing notable frameworks) to avoid hitting free-tier rate limits
    - Writes a structured flat file (fixed-width or delimited) as the handoff artifact — this is pure local disk I/O, no API calls involved, so it will not trigger rate limits regardless of repo size

2. **Mainframe emulator (Hercules or TK5)** — running locally alongside Plotoris, emulating a z/OS/MVS environment. This is where the actual JCL jobs execute, using COBOL and REXX programs as job steps.
    - Plotoris's output flat file is placed where the emulated mainframe can see it as a dataset
    - A JCL job is submitted against that dataset
    - The final report is produced on the mainframe side and passed back out

This two-system handoff (rather than one integrated app) also makes the demo easier to walk through step by step for the teacher.

---

## 3. Mapping the Four Required Operations

### Requirement 1 — Write a JCL sort

**Maps to:** Sorting the flat file Plotoris produces (e.g. by file path or detected technology category) before it moves into the matching stage.

### Requirement 2 — COBOL program that reads two files and creates a new

file containing matching records from file 1 and file 2
**Maps to:** The technology-matching step.

- File 1 = per-file detected imports/dependencies (from Plotoris)
- File 2 = reference table of known technologies (React, Express, Axios, etc.)
- COBOL reads both and writes out matches — e.g. "this file uses React" — a genuine match-merge operation, not a contrived use of COBOL.

### Requirement 3 — REXX program that removes duplicate records and sorts

remaining records by primary key
**Maps to:** The deduplication step.

- Primary key = content hash
- REXX strips out exact-duplicate files (identical hashes) and sorts what's left (e.g. by file path)
- Plays to REXX's genuine strength: lightweight string handling and table lookups, used here for the import-to-category tagging as well (e.g. tagging a file as "frontend framework" vs "HTTP client" vs "backend framework")

### Requirement 4 — JCL job: copy input file to output file using IBM

SORT utility in descending order; compare total record count before and after sort; keep output file only if counts match (success), otherwise print an error message; test both the success and failure cases
**Maps to:** The final integrity-check / report-generation step.

- Sort final report records in descending order (e.g. by how many files use a given technology)
- Verify no records were dropped during the sort — a genuine real-world mainframe safeguard, repurposed here as a data-integrity check on the final report
- Conditional logic (keep output only if step succeeds, else error) is real JCL condition-code / step-control usage, not decorative

---

## 4. Pipeline Order (proposed)

1. Plotoris scans the repo, extracts metadata, selectively calls AI for judgment calls, writes the flat file
2. JCL Step 1: Sort the flat file (Requirement 1)
3. JCL Step 2: COBOL match-merge against reference technology table (Requirement 2)
4. JCL Step 3: REXX dedupe + sort by primary key (Requirement 3)
5. JCL Step 4: IBM SORT descending + record-count integrity check + conditional keep/error (Requirement 4)
6. Final report emitted — technologies used, genuine vs. pseudo usage, duplicate file summary, record counts

Each step is a real JCL job step, with datasets passed between steps and conditional execution based on prior step return/condition codes — i.e. genuine orchestration, not just sequential scripting.

---

## 5. Why This Satisfies "Genuine Use" of JCL/COBOL/REXX

- **JCL**: real multi-step conditional orchestration, dataset passing between steps, condition-code-based branching — not just a wrapper
- **COBOL**: structured sequential record processing (file matching, merging) — its classic strength, applied to source-file metadata instead of financial/inventory records

- **REXX**: string parsing, table lookups, lightweight data manipulation — exactly what REXX was designed for, applied to deduplication and tech tagging

---

## 6. Feasibility / Difficulty Assessment

- **Plotoris preprocessing**: Low difficulty — pure JavaScript/Bun filesystem work, already largely built
- **COBOL / REXX / JCL programs**: Low-to-moderate difficulty — individually small programs (roughly a few hundred lines total across all four requirements), conceptually simple once mapped to pipeline stages
- **Main friction point**: Mainframe environment setup and JCL's strict formatting (column positions, spacing) rather than the logic itself
- AI-assisted coding (Claude Code / Google Antigravity) should handle most of the COBOL/REXX/JCL boilerplate once the design is clear

---

## 7. Local Demo Environment

- **Plotoris** runs as a normal local process (backend), performing preprocessing and writing the flat file to disk
- **Hercules or TK5** runs alongside it on the same machine, emulating a full z/OS/MVS system where the JCL/COBOL/REXX job actually executes
- Handoff: Plotoris writes output file to disk to a location visible to the Hercules environment as a dataset; JCL job is submitted against it (manually or triggered by Plotoris); final report comes back out

This pairing (Plotoris + Hercules/TK5) is confirmed workable for a local demo setup.

---

## 8. Course Context / Stated Use Case

The course teacher intends to use this project as a tool to process other students' submitted repositories and generate reports on genuine vs. superficial technology usage — essentially an automated technology-authenticity checker. Teacher also mentioned (possibly semi-jokingly) potential relevance to patentable ideas around automated detection of genuine vs. superficial technology usage in a codebase.

---

## 9. Open Items / Next Steps

- [ ] Design exact record layouts (field widths/delimiters) for the flat file Plotoris produces
- [ ] Set up Hercules or TK5 emulator environment
- [ ] Build reference technology table for COBOL match-merge step (framework names, categories, detection patterns)
- [ ] Define AI prompt(s) for genuine-vs-pseudo-usage judgment calls, with batching strategy to manage OpenRouter rate limits
- [ ] Write and test JCL Steps 1-4, including both success and failure test cases for Step 4's conditional logic
- [ ] Design final report format/output